RELATÓRIO TÉCNICO: PIPELINE CI/CD & DOCKERIZAÇÃO

Projeto: Cidades ESGInteligentes (Sustentabilidade API)
Integrante: Jenifer Lopes Ribas
Fase: 6 - DevOps e Automação

---

1. Descrição do Pipeline

O ecossistema de entrega contínua foi implementado utilizando o **GitHub Actions** como ferramenta de orquestração. A lógica do pipeline foi desenhada para garantir que nenhum código chegue em produção sem antes passar por uma validação rigorosa de integridade.

Etapas e Lógica:

1. **Build & Test (CI):** Focado na integração contínua. Compila o código Java 17 e sobe um container **Oracle XE** temporário dentro do runner para validar os testes de integração com banco de dados real.
2. **Deploy Staging (CD):** Disparado automaticamente após o sucesso da primeira etapa. Cria a imagem Docker e simula o ambiente de homologação.
3. **Deploy Production (CD):** Etapa final restrita à branch `main`. Realiza a publicação oficial do serviço, garantindo que o ambiente de produção esteja sempre sincronizado com o código estável.

---

2. Docker: Arquitetura e Comandos

A aplicação utiliza a estratégia de **Multi-stage Build**, que separa o ambiente de compilação do ambiente de execução, resultando em imagens seguras e leves.

Arquitetura do Dockerfile:

- **Stage 1 (Build):** Utiliza o `eclipse-temurin:17-jdk` para baixar dependências via Maven e gerar o arquivo `.jar`.

- **Stage 2 (Runtime):** Utiliza o `eclipse-temurin:17-jre`. Copia apenas o binário necessário para rodar, reduzindo o tamanho da imagem e a superfície de ataque.

Comandos Principais:

- **Build da Imagem:** `docker build -t sustentabilidade-api .`
- **Execução com Variáveis de Ambiente:**

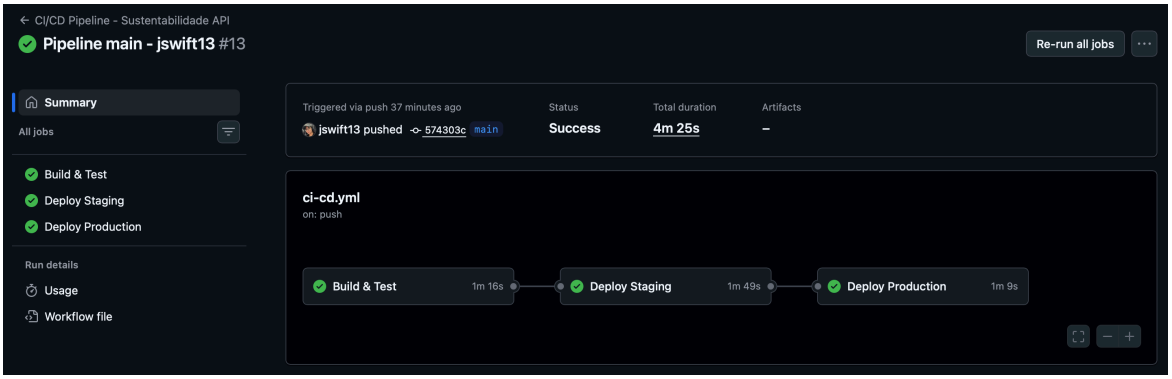
```
docker run -d -p 8080:8080 --name api-sustentabilidade \
-e SPRING_DATASOURCE_URL=jdbc:oracle:thin:@host.docker.internal:1521/XEPDE
-e SPRING_DATASOURCE_USERNAME=ESG -e SPRING_DATASOURCE_PASSWORD=
sustentabilidade-api
```

---

3. Execução do Pipeline (Build e Testes)

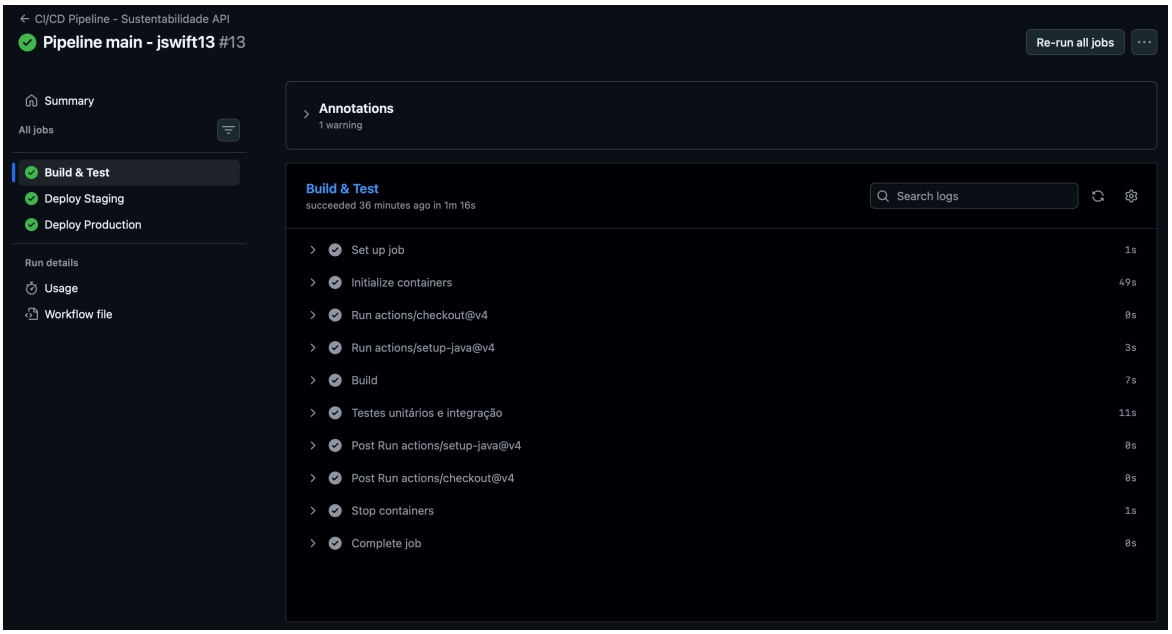
Abaixo, as evidências do sucesso da integração contínua, onde o sistema processou o build e os testes unitários/integração em menos de 2 minutos.

Visão Geral do Workflow:



.

Logs de Build e Testes:

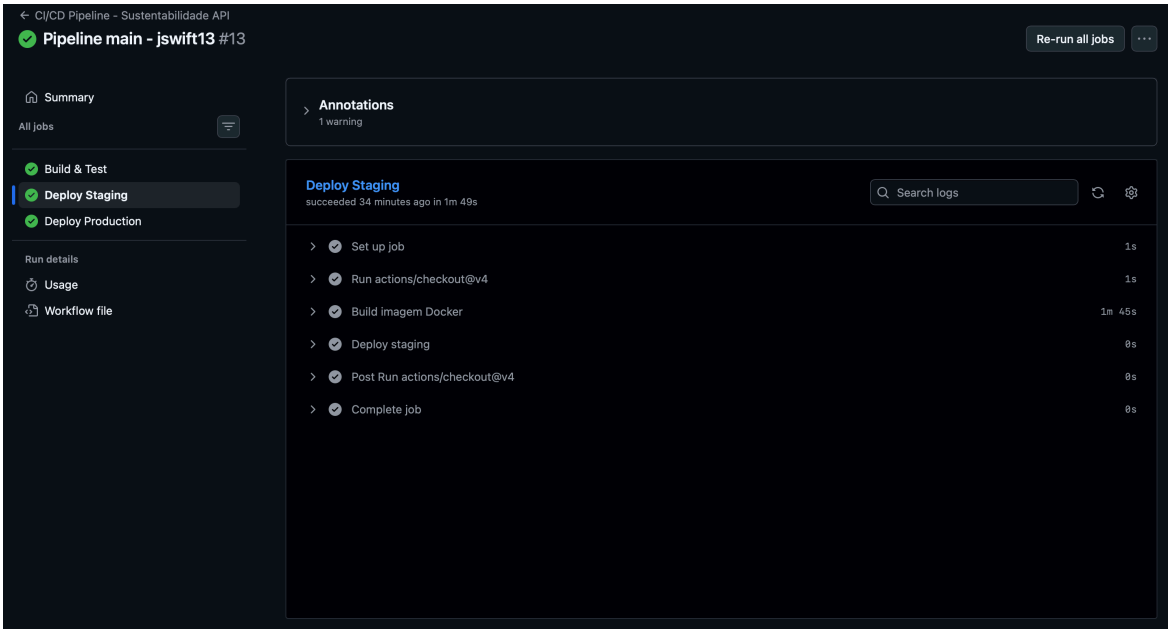


---

4. Ambientes Staging e Produção

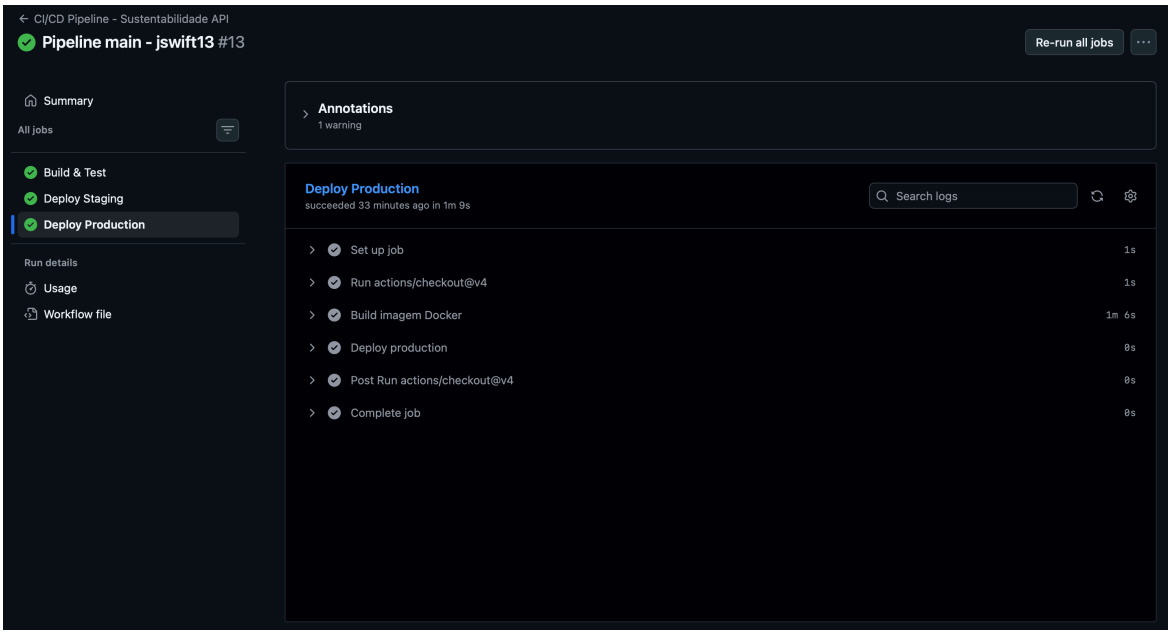
As etapas de deploy garantem a entrega da imagem Docker nos respectivos ambientes. O isolamento de portas (8080 para staging e 8081 para produção) permite testes simultâneos sem conflitos de carga.

Sucesso no Deploy Staging:



.

Sucesso no Deploy Production:



---

5. Desafios Encontrados e Soluções

Primeiramente corrigi o erro do Flyway descrito no feedback do professor na fase 4. Depois, durante a implementação, surgiram obstáculos técnicos com o banco de dados no pipeline:

Desafio: Os testes de integração falhavam porque o GitHub Actions não tinha um banco Oracle instalado.
Solução: Implementei o `services` no YAML do GitHub Actions, subindo uma imagem `gvenzl/oracle-xe` em tempo de execução, permitindo testes reais contra um banco de dados.